

Component collapse in Three-Component Tensor Decomposition for Surveillance Video Compression: An Empirical Study On VIRAT

K M Ilmun Islam Alvi

Dept. of Computer Science and Engineering
Begum Rokeya University, Rangpur
Rangpur, Bangladesh
ilmunislam101@gmail.com

I. INTRODUCTION

The data volume from surveillance cameras is still a significant challenge as they generate massive amounts of data [1]. For example, according to Statista [2], video accounted for over 80% of global internet traffic in 2023, largely driven by the growth of smart cameras, IoT devices, autonomous systems, and online video platforms. Storage is very costly, and much of the surveillance video data is redundant and static background. This redundancy is what specialized compression methods aim to exploit.

RPCA-family decomposition helps to compress surveillance video efficiently. It separates background (low-rank) from foreground (sparse) [3]. Matrix RPCA does this by flattening frames. It evolved: matrix form to tensor form to three-component form. Tensor RPCA decomposition keeps spatial-temporal structure [4] and three-component forms such as SS-RTD add a third term to capture noise or residual [5].

The technique was first introduced by Robust PCA (RPCA) [3] and was later applied to surveillance videos. Then, as videos have spatial-temporal structure, tensors were introduced to preserve it [4]. Building on this, researchers added a third component and noted that these are sensitive to trade-off parameters [6]. Separately, RPCA decomposition has been applied specifically to surveillance video compression [7].

SS-RTD was designed and validated on X-ray imaging data, where foreground is smooth. These methods also depend heavily on trade-off parameters that are difficult to set reliably [6]. Their behavior and sensitivity on surveillance video have not been systematically studied. So whether the third component helps on compressing surveillance video is an open question.

In this paper, we conduct a systematic empirical study on 180 VIRAT videos, running Tensor RPCA and SS-RTD, measuring decomposition behavior and H.264 compression. On surveillance video, we reveal that its three-component decomposition systematically collapses to two: the foreground is captured entirely by a single component, and this component is determined by the parameter values rather than by the video content.

Through this study, we make the following contributions:

- The binary collapse finding: in all 180 videos, the foreground is captured entirely by a single component while the other collapses to near zero, with no middle-ground behavior.
- The collapse winner is parameter-controlled rather than determined by scene density or content: across 20 videos and 5 configurations, the outcome was unanimous in every configuration (20/20), with $\lambda_s < \lambda_n \rightarrow S$ wins, $\lambda_s > \lambda_n \rightarrow N$ wins, and equal penalties $\rightarrow S$ wins by tie-break, identically across all densities.
- Tensor RPCA's background component L beats H.264 by approximately 6.8% on 176/180 videos, but the hybrid combination of L and SS-RTD's foreground-carrying component N does not beat H.264 on any video.

II. METHODOLOGY

A. Overview

The pipeline takes a surveillance video as input. It decomposes the video using two tensor decomposition methods. It then measures how well the resulting components compress with a standard codec. It also analyzes how one method's decomposition parameters affect the result with different values. Fig. 1 shows the overall pipeline.

B. Preprocessing

Before running anything, we preprocess the video. All the frames of the video are converted to grayscale, because color adds no benefit and triples the data. The video is also down-sampled to a lower resolution to keep the tensor operations computationally feasible. The video is normalized so that pixel values fall in a fixed range and the decompositions behave consistently throughout all the videos. Finally, the frames are stacked into a three-dimensional tensor where the two spatial dimensions correspond to frame height and width, and the third dimension corresponds to time.

C. Decomposition

The RPCA family (Robust Principal Component Analysis) decomposition methods separate a video into a low-rank part with static background and a sparse part with moving

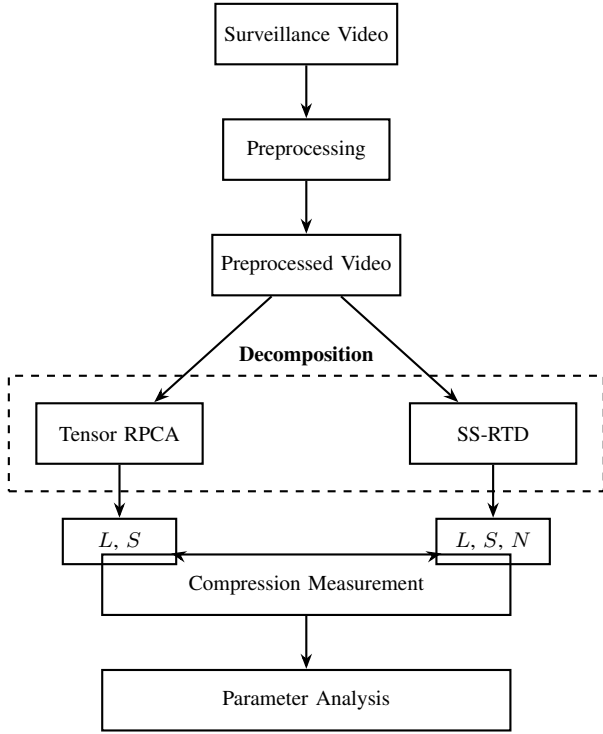


Fig. 1. Overview of the processing pipeline.

foreground. Tensor RPCA operates directly on the tensor without flattening frames into vectors, which yields L (low-rank background) and S (sparse foreground). SS-RTD adds a third component N to capture noise or residual. We use ADMM (Alternating Direction Method of Multipliers) to solve the convex optimization problems these decomposition methods pose, by dividing them into sub-problems solved in turn. ADMM updates one component while holding the others fixed, then cycles to the next, repeating until the solution converges. SS-RTD includes a penalty encouraging the foreground component to be spatially smooth, which is appropriate for its original domain (X-ray imaging) but not for surveillance video. We therefore disable this penalty, because foreground objects in surveillance video have sharp edges. The penalty tries to force smoothness onto a signal that is inherently sharp; the optimizer pushes values to extremes to satisfy an impossible constraint, and these values grow until they exceed the range representable in floating-point arithmetic, causing numerical overflow.

D. Compression Measurement

H.264 is a standard video compression codec. We encode each component with it and record the file sizes. We assess reconstruction fidelity using PSNR, measured in decibels, where higher values indicate less distortion. Structural similarity is measured using SSIM, which ranges from 0 to 1. We also measure sparsity as the fraction of near-zero values in each component.

E. Parameter Analysis

SS-RTD has trade-off parameters that control how strongly each component is penalized. These parameters directly determine which component holds the foreground data. We systematically vary these parameters and observe which component captures the foreground in different cases. This lets us understand whether SS-RTD's behavior is controlled by the parameters or by the content of the video.

III. EXPERIMENTS

A. Dataset

We use the videos-01 subset from VIRAT Ground 2.0, comprising all 180 videos, obtained from the official Kitware repository. All videos are 1280×720 and contain at least 300 frames. We use the first 300 frames of each video to limit computational cost; this cap is fixed before experimentation and applied identically to all 180 videos. All videos are converted to grayscale, resized to 320×180 , and normalized to $[0, 1]$.

B. Implementation Details

We run Tensor RPCA and SS-RTD (Smooth Sparse Robust Tensor Decomposition). Tensor RPCA computes a decomposition $L + S$, while SS-RTD computes $L + S + N$. For Tensor RPCA, the regularization parameter is set to $\lambda = 1/\sqrt{\max(H, W)}$. For SS-RTD, we use $\lambda_s = 0.01$ and $\lambda_n = 0.001$. Both methods are solved via ADMM with a maximum of 500 iterations and a convergence tolerance of 10^{-7} . We remove the smoothness penalty from SS-RTD because it causes float64 overflow on the sharp-edged foreground typical of surveillance video.

C. Compression Setup

We measure compressed file sizes by H.264 encoding, where we use ffmpeg. CRF=23 acts as the reference, and CRF=28 acts as the compressed setting. Each component is encoded separately. Each component's size is measured in kilobytes. Reconstruction quality is assessed using PSNR and SSIM. Sparsity is measured as the percentage of near-zero elements ($|x| < 10^{-6}$).

D. Parameter Sweep Setup

We characterize the collapse behavior. We conduct a sweep on 20 videos and 5 configurations. We select the 20 videos systematically. They span the foreground-density range. The four highest-density videos are reserved to ensure busy scenes are represented. The remaining sixteen are evenly spaced across the density range. The five configurations are: $(\lambda_s=0.001, \lambda_n=0.002)$; $(\lambda_s=0.002, \lambda_n=0.001)$; $(\lambda_s=0.001, \lambda_n=0.001)$ (equal); $(\lambda_s=0.01, \lambda_n=0.001)$; and $(\lambda_s=0.001, \lambda_n=0.01)$. For each video-configuration pair, we record which component wins (S or N). We also record the nonzero percentage of each component, and the reconstruction error.

TABLE I

COMPONENT NONZERO PERCENTAGE ACROSS ALL 180 VIDEOS AT THE BATCH SETTING. THE SPARSE COMPONENT S COLLAPSES TO NEAR ZERO WHILE THE RESIDUAL COMPONENT N HOLDS ALMOST ALL OF THE FOREGROUND.

Component	Min	Mean	Max
S (nonzero %)	0.02	0.147	0.56
N (nonzero %)	99.87	99.945	99.99

TABLE II

PARAMETER SWEEP RESULTS ACROSS 20 VIDEOS. FOR EACH CONFIGURATION, THE SAME COMPONENT WINS ON ALL 20 VIDEOS, WITH NO MIXED OUTCOMES.

Config	λ_s	λ_n	Relation	Winner
1	0.001	0.002	$\lambda_s < \lambda_n$	S (20/20)
3	0.001	0.001	$\lambda_s = \lambda_n$	S (20/20)
5	0.001	0.01	$\lambda_s < \lambda_n$	S (20/20)
2	0.002	0.001	$\lambda_s > \lambda_n$	N (20/20)
4	0.01	0.001	$\lambda_s > \lambda_n$	N (20/20)

E. Reproducibility

All experiments are implemented in Python 3.12 using NumPy, OpenCV, and ffmpeg. All code is publicly available at <https://github.com/IImunIslam/RPCA-Surveillance-Diagnostic>.

IV. RESULTS

A. Component Collapse

At our batch parameter setting, in all 180 videos, the foreground goes entirely to N and S collapses. S stays near zero (max 0.56% nonzero, mean 0.147%) and N stays near full (min 99.87%, mean 99.95%). None of the videos show any middle ground. One component is near 100% and the other is near 0%. This result holds across the full foreground-density range, 0.0127% to 4.263%. The collapse is not limited to empty or busy scenes. Table I reports the nonzero percentage of each component over all 180 videos.

B. Parameter Control

The parameter sweep covers 20 videos and 5 configurations. In every configuration, there are no mixed outcomes. The same component wins on all 20 videos. When $\lambda_s < \lambda_n$ or $\lambda_s = \lambda_n$, S wins; when $\lambda_s > \lambda_n$, N wins. The winner follows this parameter relationship over all densities. Table II reports the outcome for all 5 configurations over the 20 videos.

C. Compression

On 176 out of 180 videos, Tensor RPCA's background component L encoded with H.264 is smaller than the H.264 baseline. The mean reduction is 6.8%. The result is not uniform. The reduction ranges from -0.60% on the worst video to $+18.08\%$ on the best. The hybrid combination (L+N) does not beat H.264 on any of the 180 videos. It is always larger than the baseline ($1.35\times$ to $1.87\times$ the baseline size, mean $1.55\times$). Table III summarizes the compression results.

TABLE III

COMPRESSION RESULTS ACROSS ALL 180 VIDEOS. TENSOR RPCA'S BACKGROUND COMPONENT BEATS THE H.264 BASELINE ON MOST VIDEOS, WHILE THE HYBRID COMBINATION NEVER DOES.

Method	Beats H.264	Size vs. baseline
Tensor RPCA (L)	176/180	6.8% smaller (mean)
Hybrid (L+N)	0/180	$1.55\times$ larger (mean)

V. DISCUSSION

The SS-RTD's three component decomposition collapses to two on surveillance video. Which component wins depends on the set parameters, not the scene. The background of Tensor RPCA compresses better than H.264 but the hybrid does not.

A. Interpretation

The collapse means the model degenerates from three to two components. SS-RTD is designed to capture smooth foreground in S and noise in N. But on surveillance video, the foreground goes entirely to one component and the other component remains nearly empty. So the three component model (L+S+N) becomes two component model (L+ winning component) because one of the two foreground components is empty. The third component does not capture anything meaningfully distinct. SS-RTD's third component was meant to separate noise from foreground as two individual signals, but on surveillance video, they don't separate at all.

The parameter control finding means the split is not a discovery, it is an artifact, because the winner is decided completely by which penalty (λ_s or λ_n) is smaller. The assignment reflects the parameter choice, not any real trait of the video. The two foreground components are exchangeable. The component which is penalized less will absorb everything. This is why the collapse isn't a meaningful separation.

The compression result means the background can be useful but the hybrid combination doesn't help compression. L compresses better than H.264 because it is temporally stable. The hybrid fails because surveillance foreground is not sparse, it is dense. So encoding it separately as a sparse component costs more than it saves.

B. Limitations

The collapse result from the 180 videos where N wins on every video is specific to one setting ($\lambda_s=0.01$, $\lambda_n=0.001$). Because $\lambda_s > \lambda_n$, N wins every time. The parameter sweep run on 20 videos with 5 configurations partly mitigates this. We've tested only VIRAT Ground 2.0 outdoor surveillance, so our findings apply only to this kind of surveillance video. The results at full resolution or longer clips are untested. Our results are only based on grayscale, downsampled to 320×180 , capped at 300 frames. Our results for SS-RTD are without the smoothness term, we disabled it. The original method might behave differently. The compression numbers are specific to H.264, other codecs like H.265 may give different results.

C. Implications

For practitioners using these methods on surveillance video:

The third component of SS-RTD does not give any meaningful separation on surveillance video. It either holds everything or holds nothing which is decided by the parameters. So a third component provides no benefit. A simple two component based Tensor RPCA may give the same effective result with less complexity and computation. This suggests not to use the three component model on surveillance videos expecting the extra component to help, it may not help, based on our findings.

For researchers designing three-component methods: The component being controlled by the parameter ratio is a warning. When two components fight for the same signal with only given penalty weight difference between them, the split becomes an artifact. Anyone tuning these methods for surveillance should not assume the components separate meaningfully. Rather they should check for this collapse.

For the compression use case: The hybrid we tested, which combines components from different methods, does not beat a standard codec on dense foreground surveillance video. Tensor RPCA's L has the useful compression signal because it is temporally stable. There is no compression benefit in encoding the foreground as sparse when it is not actually sparse. This suggests, on this type of surveillance content, practitioners should focus on the background, not on splitting out foreground streams.

VI. CONCLUSION

We've studied Tensor RPCA and SS-RTD on 180 surveillance videos focused on decomposition behavior and compression. We found SS-RTD's three component decomposition collapses to two on surveillance video. Which component wins is controlled by the parameters not the scene, so the third component is of no benefit on this type of surveillance video. Future work can test on more datasets and video types other than VIRAT video dataset, examine whether other three component methods show the same behavior as SS-RTD, and test other codecs such as H.265.

REFERENCES

- [1] T.-T.-T. Do, Q.-T. Huynh, K. Kim, and V.-Q. Nguyen, "A survey on video big data analytics: architecture, technologies, and open research challenges," *Applied Sciences*, vol. 15, no. 14, p. 8089, 2025.
- [2] Statista Research Department, "Global internet video traffic as a share of total internet traffic worldwide from 2017 to 2023," <https://www.statista.com/statistics/871513/worldwide-video-traffic-as-share-of-internet-traffic/>, 2023, accessed: 2025-03-27.
- [3] E. J. Candès, X. Li, Y. Ma, and J. Wright, "Robust principal component analysis?" *Journal of the ACM*, vol. 58, no. 3, pp. 1–37, 2011.
- [4] C. Lu, J. Feng, Y. Chen, W. Liu, Z. Lin, and S. Yan, "Tensor robust principal component analysis: Exact recovery of corrupted low-rank tensors via convex optimization," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 5249–5257.
- [5] B. Shen, R. R. Kamath, H. Choo, and Z. Kong, "Robust tensor decomposition based background/foreground separation in noisy videos and its applications in additive manufacturing," *IEEE Transactions on Automation Science and Engineering*, vol. 20, no. 1, pp. 583–596, 2022.
- [6] Q. Yu and M. Yang, "Low-rank tensor recovery via non-convex regularization, structured factorization and spatio-temporal characteristics," *Pattern Recognition*, vol. 137, p. 109343, 2023.
- [7] C. Chen, J. Cai, W. Lin, and G. Shi, "Surveillance video coding via low-rank and sparse decomposition," in *Proceedings of the 20th ACM International Conference on Multimedia*, 2012, pp. 713–716.